

Informe del Trabajo Práctico N° 2

Integrantes:

- Gonzalez Maria Jimena
- Riffel Sheila Gabriela
- Rodriguez Maite

Pasos realizados y estructuras seleccionadas

Para resolver el problema del **triaje** de manera que se atienda a los pacientes según su nivel de riesgo de salud; siendo 1: crítico (tiene mayor prioridad), 2: moderado, 3: bajo; la estructura de datos seleccionada es una cola de prioridad implementada sobre un montículo binario de mínima. Una vez implementada la estructura integramos la cola de prioridad en la sala de emergencia y después ajustamos la cola de prioridad para considerar un segundo criterio de criticidad. Por lo tanto se definió la prioridad en base a dos criterios, que se combinan en una tupla al insertar cada paciente en la cola:

```
self.monticulo.insertar((prioridad, self.contador_llegadas, dato))
```

1. El primer criterio es el nivel de riesgo de la salud, cuanto menor sea el número de riesgo, mayor es la prioridad. Como el riesgo 1 es el mínimo valor, el montículo de mínima lo coloca en la raíz, asegurando que sea el primero en ser atendido.
2. El segundo criterio es el orden de llegada, si dos tienen el mismo nivel de riesgo, la tupla permite que el montículo compare el segundo elemento `self.contador_llegadas`, que cumple la función de contar las inserciones en la cola de prioridad. Como aumenta en 1 cada vez que se llama al método insertar, se le asigna un número único a cada paciente por orden de llegada a la cola. Entonces cuando haya pacientes con la misma prioridad, el montículo utiliza el segundo parámetro que es el orden de llegada, para comparar y decidir que se atiende primero el que menor valor tenga acumulado en el contador (análogo a ser atendido por orden de llegada).

Complejidad

Operación	Estructura	Complejidad	Justificación
Inserción	Montículo de mínima	$O(\log n)$ logarítmica	Implica agregar al final $O(1)$ y luego "filtrar hacia arriba". En el peor caso (elemento muy pequeño), el elemento debe subir desde la hoja hasta la raíz, recorriendo la altura del árbol.
Eliminación	Montículo de mínima	$O(\log n)$ (no revisa todos los n elementos, solo sube y el elemento a lo largo de la altura del árbol para reordenar la estructura)	Implica eliminar la raíz $O(1)$, mover el último elemento a la raíz $O(1)$ y luego "filtrar hacia abajo". Este proceso también recorre la altura del árbol para restablecer la propiedad del montículo.

Conclusión

La selección del Montículo Binario es óptima porque garantiza que la inserción de un nuevo paciente y la eliminación del paciente más urgente se realicen en tiempo **logarítmico** $O(\log n)$, lo cual es altamente eficiente incluso con un gran número n de pacientes en la cola de espera. Y la cola de prioridad permite al montículo considerar criterios para el ordenamiento, en este caso para el riesgo de salud y orden de llegada en caso de presentarse dos pacientes con el mismo nivel de riesgo.

Anexo

Datos tomados de una ejecución de código

Pacientes que faltan atenderse: 9

Mariela Colman -> 2-moderado (orden 20)

Jorge Rodriguez -> 3-bajo (orden 4)

Andrea Belgrano -> 3-bajo (orden 9)

Leandro García -> 3-bajo (orden 7)

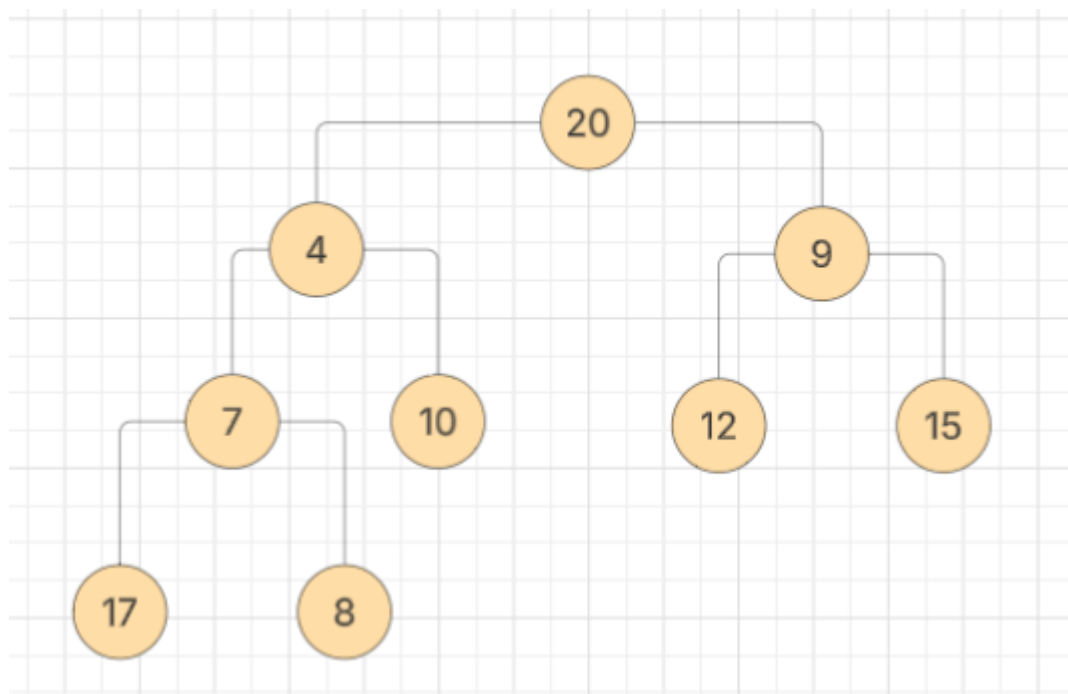
Jorge Rodriguez -> 3-bajo (orden 10)

Jorge Belgrano -> 3-bajo (orden 12)

Leandro Juarez -> 3-bajo (orden 15)

Leandro Lopez -> 3-bajo (orden 17)

Agustina García -> 3-bajo (orden 8)



Gráfica para verificar el correcto funcionamiento del montículo, como vemos siempre respeta que los hijos son mayores que los padres, y la raíz es la menor de todas (por ser de riesgo 2)